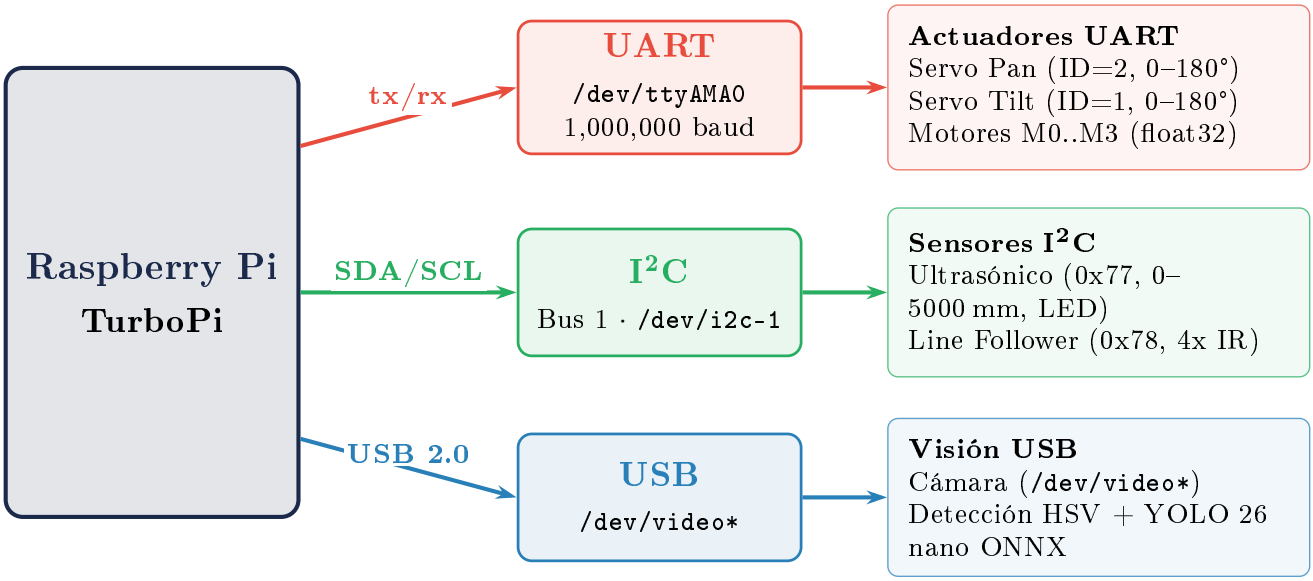
Mapa de Buses de Comunicación

TurboPi Hiwonder — Robot de Fútbol Autónomo

Fuente: `hardware.py` · Abril 2026



Resumen de Buses

Bus	Dispositivo	Dirección	Baud/Reg	Función
UART	SerialBus	`/dev/ttyAMA0`	1,000,000	2 servos pan/tilt + 4 motores DC omnidireccionales
I²C	Ultrasónico	`0x77`	Reg `0x02` LED	Sensor distancia 0–5000 mm + LED RGB
I²C	Line Follower	`0x78`	Reg `0x01`	4 sensores IR digitales (bitmask)
USB	Cámara	`/dev/video*`	—	Detección HSV adaptativa + YOLO 26 nano ONNX

UART — `/dev/ttyAMA0`

Protocolo binario

Todas las tramas usan header fijo `0xAA 0x55`, seguido de tipo, longitud, datos y CRC8 (tabla de 256 bytes).

Campo	Bytes	Descripción
Header	`0xAA 0x55`	Sync bytes fijos
Tipo	1	`0x04`=servo, `0x03`=motor
Longitud	1	Cantidad de bytes de datos
Datos	N	Ver estructura abajo
CRC8	1	Sobre campos tipo+longitud+datos

Estructura de trama Servo (tipo `0x04`)

`dur_ms(2B LE)` + `count(1B)` + `[id(1B)` + `pos_pulse(2B LE)]` × count.

Servo Pan (ID=2): centro=86°, rango 0–180°, **invertido**. Pulso: $p = 500 + (\theta/180) \times 2000\ \mu s$.
Servo Tilt (ID=1): centro=129°, rango 0–180°, **invertido**. Mismo cálculo de pulso.

Estructura de trama Motor (tipo `0x03`)

`0x05(1B)` + `count(1B=4)` + `[motor_id(1B)` + `speed_float32(4B LE)]` × 4.

Motor 0 = delantero-izq, Motor 1 = delantero-der, Motor 2 = trasero-izq, Motor 3 = trasero-der.
Velocidad: `float32` con signo (negativo=atrás). Mapeo diferencial: $(-v_L,\ v_R,\ -v_L,\ v_R)$ con cap=250.

I²C — Bus 1

Ambos sensores comparten el bus con un `threading.Lock` para acceso seguro.

Sensor Ultrasónico (`0x77`)

- **Distancia:** write `[0x00]`, luego read 2 bytes como `uint16_little_endian`. Rango 0–5000 mm. Error → 9999.
- **LED RGB:** registro `0x02`. Datos: `[mode, R, G, B, R, G, B]`. `mode=0x00` sólido, `0x01` parpadeo.

Line Follower (`0x78`)

- **Lectura:** registro `0x01`, 1 byte. Bitmask: bit0=IR1, bit1=IR2, bit2=IR3, bit3=IR4.
- **Calibración:** 20 muestras para establecer baseline (pasto dominante = `True`).
- **Detección:** ≥2 sensores cambian respecto al baseline ⇒ línea blanca detectada.

Cámara USB

- Resolución 320×240, buffer=1, auto-exposición deshabilitada, exposición manual 150–450 (default 200).
- **HSV adaptativo:** H naranja = 0–10 y 168–179, S≥230, V≥20. Gamma dinámico 1.0–2.0. Modo estricto/relajado/r
  adquisición.
- **Filtros:** area>700px, radio>15px, circularidad>0.20. Centro de hue adaptativo con EMA α=0.15.
- **YOLO 26 nano ONNX:** modelo ONNX 320×320, confianza≥0.35, clase 0 = pelota.

Ejemplos Python

UART: enviar comando servo + motor

```python
import struct, serial

CRC8 = [0,94,188,226,97,63,221,131,194,156,126,32,163,
  253,31,65,157,195,33,127,252,162,64,30,95,1,227,189,
  62,96,130,220,35,125,159,193,66,28,254,160,225,191,
  93,3,128,222,60,98,190,224,2,92,223,129,99,61,124,
  34,192,158,29,67,161,255,70,24,250,164,39,121,155,
  197,132,218,56,102,229,187,89,7,219,133,103,57,186,
  228,6,88,25,71,165,251,120,38,196,154,101,59,217,
  135,4,90,184,230,167,249,27,69,198,152,122,36,248,
  166,68,26,153,199,37,123,58,100,134,216,91,5,231,
  185,140,210,48,110,237,179,81,15,78,16,242,172,47,
  113,147,205,17,79,173,243,112,46,204,146,211,141,
  111,49,178,236,14,80,175,241,19,77,206,144,114,44,
  109,51,209,143,12,82,176,238,50,108,142,208,83,13,
  239,177,240,174,76,18,145,207,45,115,202,148,118,
  40,171,245,23,73,8,86,180,234,105,55,213,139,87,9,
  235,181,54,104,138,212,149,203,41,119,244,170,72,
  22,233,183,85,11,136,214,52,106,43,117,151,201,74,
  20,246,168,116,42,200,150,21,75,169,247,182,232,10,
  84,215,137,107,53]

def crc8(data):
    c = 0
    for b in data:
        c = CRC8[c ^ b]
    return c

def pulse(angle):
    return int(500 + (max(0, min(180, angle)) / 180.0) * 2000)

def burst(ser, pan, tilt, dur_ms, m1, m2, m3, m4):
    pp, tp = pulse(pan), pulse(tilt)
    d = int(dur_ms)
    sd = bytearray([0x01, d & 0xFF, (d >> 8) & 0xFF, 2,
      2, pp & 0xFF, (pp >> 8) & 0xFF,
      1, tp & 0xFF, (tp >> 8) & 0xFF])
    fs = bytearray(b"\xaa\x55") + bytes([0x04, len(sd)]) + sd
    fs.append(crc8(fs[2:]))
    md = bytearray([0x05, 4])
    for mid, val in ((0, m1), (1, m2), (2, m3), (3, m4)):
        md += struct.pack("<Bf", mid, float(val))
    fm = bytearray(b"\xaa\x55") + bytes([0x03, len(md)]) + md
    fm.append(crc8(fm[2:]))
    ser.write(fs + fm)

ser = serial.Serial("/dev/ttyAMA0", 1_000_000)
burst(ser, pan=86, tilt=129, dur_ms=500, m1=200, m2=200, m3=0, m4=0)
ser.close()
```

I²C: leer ultrasónico + line follower

```python
from smbus2 import SMBus, i2c_msg

bus = SMBus(1)

# --- Ultrasonico (0x77): leer distancia mm ---
w = i2c_msg.write(0x77, [0])
bus.i2c_rdwr(w)
r = i2c_msg.read(0x77, 2)
bus.i2c_rdwr(r)
dist_mm = int.from_bytes(bytes(list(r)), "little")
```

```python
print(f"Distancia: {dist_mm} mm")

# --- Ultrasonico: LED verde solido ---
bus.write_i2c_block_data(0x77, 0x02, [0x00, 0x00, 0x10, 0x00,
                                       0x00, 0x10, 0x00])

# --- Line Follower (0x78): 4 sensores IR ---
v = bus.read_byte_data(0x78, 0x01)
sensors = [bool(v & b) for b in (0x01, 0x02, 0x04, 0x08)]
print(f"Sensores IR: {sensors}")
bus.close()
```

Cámara: detección HSV naranja

```python
import cv2, numpy as np

cap = cv2.VideoCapture(0)
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 320)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 240)
cap.set(cv2.CAP_PROP_AUTO_EXPOSURE, 1)
cap.set(cv2.CAP_PROP_EXPOSURE, 200)

for _ in range(100):
    ok, frame = cap.read()
    if not ok:
        continue
    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
    m1 = cv2.inRange(hsv, (0, 230, 20), (10, 255, 255))
    m2 = cv2.inRange(hsv, (168, 230, 20), (179, 255, 255))
    mask = cv2.bitwise_or(m1, m2)
    k = np.ones((3, 3), np.uint8)
    mask = cv2.dilate(cv2.morphologyEx(mask, cv2.MORPH_OPEN, k), k)
    contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
                                   cv2.CHAIN_APPROX_SIMPLE)
    for c in contours:
        (x, y), r = cv2.minEnclosingCircle(c)
        if cv2.contourArea(c) > 700 and r > 15:
            print(f"Pelota ({int(x)}, {int(y)}) radio={r:.0f}")
            break
cap.release()
```

Ejemplos Rust

Dependencias (`Cargo.toml`):

```
[dependencies]
serialport = "4"
i2cdev = "0.6"
opencv = { version = "0.93", features = ["clang-runtime"] }
```

UART: enviar comando servo + motor

```rust
use serialport::SerialPort;
use std::io::Write;

const CRC8: [u8; 256] = [0,94,188,226,97,63,221,131,
  194,156,126,32,163,253,31,65,157,195,33,127,252,
  162,64,30,95,1,227,189,62,96,130,220,35,125,159,
  193,66,28,254,160,225,191,93,3,128,222,60,98,190,
  224,2,92,223,129,99,61,124,34,192,158,29,67,161,
  255,70,24,250,164,39,121,155,197,132,218,56,102,
  229,187,89,7,219,133,103,57,186,228,6,88,25,71,
  165,251,120,38,196,154,101,59,217,135,4,90,184,
  230,167,249,27,69,198,152,122,36,248,166,68,26,
  153,199,37,123,58,100,134,216,91,5,231,185,140,
  210,48,110,237,179,81,15,78,16,242,172,47,113,
  147,205,17,79,173,243,112,46,204,146,211,141,
  111,49,178,236,14,80,175,241,19,77,206,144,114,
  44,109,51,209,143,12,82,176,238,50,108,142,208,
  83,13,239,177,240,174,76,18,145,207,45,115,202,
  148,118,40,171,245,23,73,8,86,180,234,105,55,
  213,139,87,9,235,181,54,104,138,212,149,203,41,
  119,244,170,72,22,233,183,85,11,136,214,52,106,
  43,117,151,201,74,20,246,168,116,42,200,150,21,
  75,169,247,182,232,10,84,215,137,107,53];

fn crc8(data: &[u8]) -> u8 {
    let mut c: u8 = 0;
    for &b in data { c = CRC8[(c ^ b) as usize]; }
    c
}

fn pulse(angle: u16) -> u16 {
    (500.0 + (angle.min(180) as f32 / 180.0) * 2000.0) as u16
}

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let mut port = serialport::new("/dev/ttyAMA0", 1_000_000)
        .timeout(std::time::Duration::from_millis(100))
        .open()?;
    let (pan, tilt, dur) = (86u16, 129u16, 500u16);
    let (pp, tp) = (pulse(pan), pulse(tilt));
    let sd: Vec<u8> = vec![
        0x01, dur as u8, (dur >> 8) as u8, 2,
        2, pp as u8, (pp >> 8) as u8,
        1, tp as u8, (tp >> 8) as u8];
    let mut fs: Vec<u8> = vec![0xAA, 0x55, 0x04, sd.len() as u8];
    fs.extend(&sd);
    fs.push(crc8(&fs[2..]));
    let mut md: Vec<u8> = vec![0x05, 4];
    for (id, val) in [(0u8, 200f32), (1, 200.0), (2, 0.0), (3, 0.0)] {
        md.push(id);
        md.extend(&val.to_le_bytes());
    }
    let mut fm: Vec<u8> = vec![0xAA, 0x55, 0x03, md.len() as u8];
    fm.extend(&md);
    fm.push(crc8(&fm[2..]));
    port.write_all(&[&fs[..], &fm[..]].concat())?;
```

```
    println!("Trama enviada: {} bytes", fs.len() + fm.len());
    Ok(())
}
```

I²C: leer ultrasónico + line follower

```
use i2cdev::core::*;
use i2cdev::linux::LinuxI2CDevice;

fn read_ultra(dev: &mut LinuxI2CDevice) -> Result<u16, ()> {
    dev.write(&[0x00]).map_err(|_| ())?;
    let mut buf = [0u8; 2];
    dev.read(&mut buf).map_err(|_| ())?;
    Ok(u16::from_le_bytes(buf).min(5000))
}

fn set_led(dev: &mut LinuxI2CDevice, r: u8, g: u8, b: u8)
    -> Result<(), ()> {
    dev.write(&[0x02, 0x00, r, g, b, r, g, b]).map_err(|_| ())
}

fn read_line(dev: &mut LinuxI2CDevice) -> Result<[bool; 4], ()> {
    dev.write(&[0x01]).map_err(|_| ())?;
    let mut buf = [0u8; 1];
    dev.read(&mut buf).map_err(|_| ())?;
    let v = buf[0];
    Ok([v & 1 != 0, v & 2 != 0, v & 4 != 0, v & 8 != 0])
}

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let mut ultra = LinuxI2CDevice::new("/dev/i2c-1", 0x77)?;
    let mut line = LinuxI2CDevice::new("/dev/i2c-1", 0x78)?;
    let dist = read_ultra(&mut ultra)?;
    println!("Distancia: {} mm", dist);
    set_led(&mut ultra, 0x00, 0x10, 0x00)?;
    let sensors = read_line(&mut line)?;
    println!("Sensores IR: {:?}", sensors);
    Ok(())
}
```

Cámara: detección HSV naranja

```
use opencv::prelude::*;
use opencv::{imgproc, videoio, core};

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let mut cap = videoio::VideoCapture::new(0, videoio::CAP_ANY)?;
    cap.set(videoio::CAP_PROP_FRAME_WIDTH, 320.0)?;
    cap.set(videoio::CAP_PROP_FRAME_HEIGHT, 240.0)?;
    let mut frame = Mat::default();
    for _ in 0..100 {
        cap.read(&mut frame)?;
        if frame.empty() { continue; }
        let mut hsv = Mat::default();
        imgproc::cvt_color(&frame, &mut hsv,
            imgproc::COLOR_BGR2HSV, 0)?;
        let mut m1 = Mat::default();
        let mut m2 = Mat::default();
        core::in_range(&hsv,
            &core::Scalar::new(0.0, 230.0, 20.0, 0.0),
            &core::Scalar::new(10.0, 255.0, 255.0, 0.0),
            &mut m1)?;
        core::in_range(&hsv,
            &core::Scalar::new(168.0, 230.0, 20.0, 0.0),
            &core::Scalar::new(179.0, 255.0, 255.0, 0.0),
```

```
            &mut m2)?;
        let mut mask = Mat::default();
        core::bitwise_or(&m1, &m2, &mut mask,
            &core::no_array())?;
        let mut contours =
            opencv::types::VectorOfVectorOfPoint::new();
        imgproc::find_contours(&mut mask, &mut contours,
            imgproc::RETR_EXTERNAL,
            imgproc::CHAIN_APPROX_SIMPLE,
            core::Point::new(0, 0))?;
        for c in contours.iter() {
            let area = imgproc::contour_area(&c, false)?;
            let ((x, y), r) = imgproc::min_enclosing_circle(&c)?;
            if area > 700.0 && r > 15.0 {
                println!("Pelota ({:.0}, {:.0}) radio={:.0}", x, y, r);
                break;
            }
        }
    }
    Ok(())
}
```